Revision 28–31 · Inside a serving engine: why, API, metrics & the request path

Interactive revision of lesson 28, lesson 29, lesson 30 and lesson 31: one question at a time, logged in my own reasoning.

Q1 · "We need a bigger GPU" SOLID

Covers: 16, 28

QUESTION

A FastAPI endpoint calling `model.generate()` is great with one user. With 20 users the median wait for the first token is ~40 s, yet `nvidia-smi` shows the GPU's math units mostly idle. A teammate says "we need a bigger GPU". What's actually wrong, and why would a bigger GPU barely help?

Answer

- There's no batching: each `generate()` call owns the GPU until its whole reply is finished, so requests run **one at a time** and each user waits behind everyone ahead of them.
- The GPU looks idle because decode is **memory-bandwidth-bound**: every step reads all the weights to produce a single token, so the math units have almost nothing to do.
- A bigger GPU only makes each step somewhat faster (more bandwidth); the queue is still serial, so user 20 still waits 19 full replies.
- The fix is batching, done properly: **continuous batching**, so one weight read serves many requests at once.

RE-DRILL

Re-drill: put a number on it (6.4 s replies → user 10 waits ≈ 58 s) and say what a 1.5× faster GPU changes (≈ 39 s: still unusable).

Q2 · Contiguous vs paged KV memory PARTIAL

Covers: 19, 28

QUESTION

A plain server reserves 4,096 tokens of KV cache per request, contiguously, up front. Real requests average ~900 tokens. What fraction of the reservation holds tokens, and roughly how many more requests fit with 16-token pages?

How I solved it

1. Contiguous: used ÷ reserved = 900 ÷ 4,096 ≈ **22%**, inside the 20–38% the vLLM team measured in earlier systems.
2. Paged: 900 ÷ 16 = 56.25, rounded up to **57 blocks = 912 tokens**; at most one partly filled block is wasted.
3. Compare footprints: 4,096 ÷ 912 ≈ **4.5×** more requests in the same memory.

> **RE-DRILL**
>
> Re-drill: same question with a 32,768-token max and 2,000-token average (2,000 ÷ 32,768 ≈ 6% used; ≈ 16× more requests).

Q3 · Three support tickets, all on the wire PARTIAL

Covers: 29

> **QUESTION**
>
> Tickets on an OpenAI-compatible endpoint: (a) answers read like the model is continuing someone else's chat; (b) answers often stop mid-sentence; (c) the streaming usage dashboard shows zero tokens. For each, what do you look for on the wire, and what's the fix?

Answer

Ticket	Look at	Cause	Fix
(a) continues someone else's chat	The tokens the server built; `usage.prompt_tokens` vs expected	Wrong or missing **chat template**: roles never become proper control tokens, no `assistant` cue	Correct template (`--chat-template` if the model lacks one)
(b) stops mid-sentence	**`finish_reason`**	`length`: hit `max_tokens` (an end token would show `stop`)	Raise `max_tokens`
(c) usage shows zero when streaming	Last chunks before `[DONE]`	No usage chunk: `stream_options.include_usage` not set	Set it (or force usage server-side)

> **RE-DRILL**
>
> Re-drill: what `finish_reason` would a stop string give, and what would a function call give? (`stop`, `tool_calls`.)

Q4 · Reading two scrapes PARTIAL

Covers: 22, 30

> **QUESTION**
>
> Two scrapes of `/metrics`, 60 s apart: `generation_tokens_total` 1,200,000 → 1,380,000; `request_success_total{length}` 3,000 → 3,350; `{stop}` 9,000 → 9,050; `num_requests_waiting` 0 and 0. What was the generation throughput, did anyone queue, and what does the `finished_reason` split say?

How I solved it

1. Counters only mean something as a difference: (1,380,000 − 1,200,000) ÷ 60 = 180,000 ÷ 60 = **3,000 tok/s** over that minute.
2. `num_requests_waiting` is a gauge, a snapshot at two instants; a queue could have formed and drained in between. **Can't conclude nobody queued.** The queue-time histogram (`vllm:request_queue_time_seconds`) records every request and would answer it.
3. Differences again: 350 requests ended `length`, 50 ended `stop`, so **87.5%** of this minute's requests hit `max_tokens`, versus 25% over the server's lifetime (3,000 of 12,000).
4. So this minute is unusual: either a load test forcing output length (`ignore_eos` + fixed `max_tokens`) or a client truncating real users with a low `max_tokens`. Check before quoting the 3,000 tok/s. (180,000 ÷ 400 ≈ 450 tokens per request.)

> **RE-DRILL**
>
> Re-drill: from two TTFT histogram scrapes, recover the window's mean (Δsum ÷ Δcount) and its P50 bracket.

Q5 · Blocks, block table and slots PARTIAL

Covers: 17, 27, 31

> **QUESTION**
>
> A request arrives with a 40-token prompt; blocks hold 16 tokens; the cache manager gives it blocks 5, 12 and 2, in order. Why exactly 3 blocks? Which slot does position 35 write to? At position 48 it needs a 4th block but the pool is empty: what happens, and who pays?

How I solved it

1. Blocks = 40 ÷ 16 = 2.5, rounded up = **3**.
2. Position 35: logical block = 35 ÷ 16 = **2** remainder **3**. The block table maps logical 0, 1, 2 → physical 5, 12, 2, so it's physical **block 2**, offset 3.
3. Slot = block × 16 + offset = 2 × 16 + 3 = **35** (equal to the position only because the third block happens to be block 2; with block 9 it would be 147).
4. Position 48 starts logical block 3, but the pool is empty, so vLLM **preempts** a running request: discards its cache and later recomputes prompt + generated tokens. The evicted request stalls, and the recompute steals step time from everyone.

RE-DRILL

Re-drill: with blocks [8, 1, 30], find the slots for positions 0, 17 and 40 (128, 17, 488).

Q6 · Which budget keeps a request out? MISSED

Covers: 27, 30, 31

QUESTION

`max_num_seqs = 256`, `max_num_batched_tokens = 8192`, and 256 requests are decoding. A new 500-token prompt arrives. Does it join the next step, which budget decides, which `/metrics` line shows it, and what has to happen for it to get in?

Answer

- Token budget: the decodes use 256 tokens, leaving 7,936; the 500-token prompt would fit.
- Sequence budget: 256 of 256 is **full**, so it does **not** join. Both budgets must have room; here `max_num_seqs` decides.
- It sits in the waiting queue: `vllm:num_requests_waiting` +1 (if a scrape catches it), and its wait is recorded in `vllm:request_queue_time_seconds` once it runs. Its TTFT includes the wait.
- It gets in when a running request finishes (or is preempted), freeing a sequence slot and its blocks; the next step schedules decodes first, then admits it for prefill.
- If this is common, `max_num_seqs` is the bottleneck while the token budget idles. Raise it only if the cache holds that many requests at final length, or you trade queueing for preemption.

RE-DRILL

Re-drill: same engine, 100 running requests and a 9,000-token prompt: which budget blocks it now? (Tokens: 100 + 9,000 > 8,192.)